Paid Stories — RevenueCat End-to-End Implementation Plan

Overview

This implementation enables premium story purchases across iOS and Android using RevenueCat as the centralized purchase orchestration layer.

RevenueCat will manage:
• Apple App Store purchases
• Google Play Billing purchases
• Receipt validation
• Purchase lifecycle events
• Refund/revocation synchronization
• Entitlement management
• Purchase restoration

The backend will focus on:
• Story access control
• Purchase persistence
• User entitlement mapping
• Notifications
• Admin tooling
• Analytics/auditing

This architecture removes the need for direct StoreKit verification, Google Play verification, Apple server notifications, and Google RTDN infrastructure.

Scope

Included:
• iOS premium story purchases
• Android premium story purchases
• RevenueCat SDK integration
• RevenueCat webhook integration
• Purchase restoration
• Refund/revocation handling
• Backend entitlement synchronization
• Push notifications
• Email receipts
• Admin purchase management

Architecture Overview

Frontend App
↓
RevenueCat SDK
↓
Apple App Store / Google Play
↓
RevenueCat
↓
Backend Webhooks + APIs
↓
Story Access System

Purchase Flow

1. User opens premium story
2. App fetches product metadata from backend
3. RevenueCat SDK loads store product
4. User completes native checkout
5. RevenueCat validates purchase with Apple/Google
6. RevenueCat emits webhook event to backend
7. Backend records purchase
8. Existing access-control system unlocks story
9. User receives push notification + receipt email

Refund Flow

1. User requests refund from Apple/Google
2. RevenueCat receives store lifecycle update
3. RevenueCat sends cancellation webhook
4. Backend revokes purchase
5. Story access automatically removed
6. User receives revocation notification

Frontend Implementation

SDK

Frontend SDK:
• RevenueCat Purchases SDK

Platform SDKs:
• iOS → RevenueCat iOS SDK
• Android → RevenueCat Android SDK
• React Native → `react-native-purchases`

Frontend Responsibilities

Authentication Sync

On login:
• Initialize RevenueCat
• Associate RevenueCat user with backend authenticated user ID

Flow:
• User logs into app
• App calls:
`Purchases.logIn(userId)`

Purpose:
• RevenueCat purchases become tied to backend users
• Enables secure entitlement synchronization

Product Loading

Frontend flow:

1. Fetch stories from backend
2. Fetch RevenueCat offerings/products
3. Match story to RevenueCat product ID

Backend response example:
• story_id
• revenuecat_product_id
• title
• price
• is_premium

Purchase Flow

Frontend actions:

1. User taps “Unlock Story”
2. RevenueCat SDK launches native purchase UI
3. Apple/Google handles payment
4. RevenueCat validates automatically
5. SDK returns updated customer info
6. Frontend refreshes story access from backend

Frontend no longer sends:
• Apple receipts
• Google purchase tokens

RevenueCat abstracts all payment validation.

Restore Purchases

Frontend actions:
• Call:
`restorePurchases()`

Used for:
• Reinstall recovery
• New device sync
• Account restoration

Post-restore:
• Frontend refreshes backend entitlements

Offline Recovery

Frontend should:
• Retry syncing purchases on app startup
• Refresh customer info after reconnect
• Refresh story access periodically

Backend Implementation

Core Responsibilities

Backend responsibilities:
• Receive RevenueCat webhooks
• Record purchases
• Grant/revoke story access
• Expose entitlement APIs
• Maintain purchase history
• Trigger notifications

RevenueCat becomes the source of truth for payment validation.

Files to Create

Payments Module

Path	Purpose
`app/modules/payments/__init__.py`	Payments module
`app/modules/payments/models/__init__.py`	Model exports
`app/modules/payments/models/revenuecat_event.py`	Webhook dedupe/audit table
`app/modules/payments/services/__init__.py`	Service exports
`app/modules/payments/services/revenuecat_service.py`	RevenueCat webhook + API logic
`app/modules/payments/services/purchase_service.py`	Purchase grant/revoke logic
`app/modules/payments/router/payments.py`	User payment endpoints
`app/modules/payments/router/webhooks.py`	RevenueCat webhook endpoint
`app/modules/admin/router/admin_purchases.py`	Admin purchase management

`app/tasks/payments.py`	Email/push notification tasks
`alembic/versions/add_revenuecat_fields.py`	Database migration
`tests/payments/test_webhooks.py`	Webhook handling tests
`tests/payments/test_access.py`	Purchase access tests
`tests/payments/test_revocation.py`	Refund/revoke tests

Files to Modify

Path	Change
`app/config.py`	Add RevenueCat configuration
`app/main.py`	Register payment/webhook routers
`app/celery_app.py`	Register payment tasks
`app/models.py`	Register new models
`app/modules/story/models/story_entity.py`	Add RevenueCat product fields

`app/modules/user/models/user_story_purchase.py`	Add RevenueCat purchase metadata
`app/modules/story/services/story_access.py`	Add revoked purchase filtering
`app/modules/admin/router/admin_stories.py`	Add RevenueCat product mapping fields
`pyproject.toml`	Add RevenueCat SDK/client dependencies

Database Changes

Story Table

New fields:
- `revenuecat_product_id`
- `currency`

Purpose:
- Maps stories to RevenueCat products

Purchase Table

New fields:
- `platform` (`apple` / `google`)
- `revenuecat_transaction_id`
- `revenuecat_product_id`
- `store_transaction_id`
- `purchased_at`
- `revoked_at`
- `revoke_reason`

Webhook Event Table

New table:
`revenuecatevent`

Fields:
• event_id
• event_type
• received_at
• processed_at
• payload

Purpose:
• Deduplicate webhook retries
• Audit logging
• Event tracing

Backend APIs

GET `/payments/products`

Purpose:
Returns purchasable story products.

Response:
• story_id
• title
• revenuecat_product_id
• price
• currency
• is_premium

Frontend usage:
• Match stories to RevenueCat offerings/products

GET `/payments/me/purchases`

Returns:
• Purchase history
• Platform source
• Purchase status
• Revocation status

Used for:
• Restore flows

• Purchase history UI
• Debugging/account support

RevenueCat Webhook Integration

Webhook Endpoint

POST `/payments/webhooks/revenuecat`

RevenueCat sends lifecycle events directly to this endpoint.

Authentication

Webhook verification:
• RevenueCat authorization header
• Webhook secret validation

Webhook Events to Handle

Purchase Events

Examples:
• INITIAL_PURCHASE
• NON_RENEWING_PURCHASE

Backend actions:
• Record purchase
• Grant story access
• Trigger receipt email
• Trigger push notification

Refund / Cancellation Events

Examples:
• CANCELLATION
• EXPIRATION

Backend actions:
• Mark purchase revoked
• Remove story access
• Notify user

Restore Events

Examples:
• RESTORE
• TRANSFER

Backend actions:
• Re-sync entitlements
• Restore story access

Purchase Recording Logic

Purchase service responsibilities:
• Idempotent purchase creation
• Purchase ownership validation
• Revocation handling
• Entitlement updates

Duplicate Protection

Deduplication keys:
• RevenueCat transaction ID
• RevenueCat event ID

Purpose:
• Prevent duplicate purchases
• Handle webhook retries safely

Access Control Updates

Existing access-control architecture remains intact.

Enhancement:
• Access checks must ignore revoked purchases

Logic:
• Active purchase → access granted
• Revoked/refunded purchase → access denied

No changes required to existing HTTP 402 enforcement.

Admin Features

Admin capabilities:
• Configure RevenueCat product IDs
• View purchases
• Filter by:
• platform
• revoked status
• story
• user

• Revoke access manually
• Audit webhook events

Manual revocation:
• Removes backend access only
• Does not issue Apple/Google refunds

Notifications & Side Effects

Successful Purchase

Triggers:
• Email receipt
• Push notification
• “Story unlocked” confirmation

Refund/Revoke

Triggers:
• Access revoked notification
• Refund/access-change email

Background Tasks

Asynchronous jobs:
• Purchase receipts
• Refund notifications
• Push notifications
• Analytics events

RevenueCat Configuration

RevenueCat Dashboard Setup

Required setup:
• Connect App Store app
• Connect Google Play app
• Create products
• Configure entitlements
• Configure webhook endpoint
• Generate API keys

Product Structure

Recommended:
• One RevenueCat product per story

Examples:
• `story_dark_knight`
• `story_lost_signal`

RevenueCat maps:
• Apple product IDs
• Google product IDs

Infrastructure & Environment Variables

Backend Config

Examples:
• `REVENUECAT_API_KEY`
• `REVENUECAT_WEBHOOK_SECRET`

Frontend Config

Examples:
• RevenueCat public SDK keys
• Environment-specific app keys

Security Model

RevenueCat handles:
• Receipt validation
• Store verification
• Refund synchronization

Backend security responsibilities:
• Verify webhook authenticity
• Validate RevenueCat app user ID
• Enforce authenticated ownership
• Prevent duplicate entitlements

Testing Strategy

Automated Tests

Coverage:
• Webhook processing
• Purchase recording
• Revocation flow
• Duplicate webhook handling
• Access synchronization

Manual Testing

iOS:
• Sandbox purchases
• Refund testing

Android:
• Internal testing purchases
• Refund testing

Cross-platform:
• Restore purchases
• Account switching
• Reinstall recovery

Expected Outcome

After implementation:

Users will be able to:
• Purchase premium stories natively on iOS and Android
• Restore purchases across devices
• Instantly unlock premium content
• Automatically lose access after refunds/revocations

The platform will support:
• Centralized payment orchestration via RevenueCat
• Simplified payment infrastructure
• Reliable entitlement synchronization
• Automated refund handling
• Secure webhook-driven access control
• Scalable future support for subscriptions and bundles